



Laboratório de Programação Frontend



Universidade de Vassouras



Prof. Rodrigo Ximenes



Javascript — Callbacks, Promises e Async/Await



Conteúdo da Aula

- Revisão rápida de JavaScript
- Callbacks
- Callback Hell e soluções
- Promises
- Async / Await
- Programação assíncrona e chamadas ao backend
- Tratamento de exceções
- Exercícios do básico ao avançado



Revisão rápida de JavaScript

Antes de mergulhar em callbacks e Promises, vamos revisar os fundamentos:

1 Variáveis

```
// let e const respeitam escopo de bloco
if(true){
  let x = 10;      // variável que só existe dentro do bloco
  const y = 20;    // constante que só existe dentro do bloco
  console.log(x, y); // 10 20
}
// console.log(x, y); // ERRO: x e y não existem fora do bloco
```

```
// var ignora escopo de bloco
if(true){
  var z = 30;
  console.log(z); // 30
}
console.log(z); // 30 → pode causar bugs se não quiser que z exista fora do bloco
```

2 Tipos de dados

```
let texto = "Olá, mundo!"; // string
let numero = 42;           // number
let ligado = true;         // boolean
let lista = [1,2,3];       // array
let pessoa = {nome:"Ana", idade:20}; // objeto
```

3 Funções

```
function soma(a, b){
  return a + b; // retorna a soma
}
const multiplicar = (a,b) => a*b; // arrow function
console.log(soma(2,3));           // 5
console.log(multiplicar(2,3));    // 6
```

4 Eventos

```
const botao = document.querySelector("button");
botao.addEventListener("click", () => {
  alert("Você clicou no botão!"); // executa quando o botão é clicado
});
```

5 DOM

```
const titulo = document.getElementById("titulo");
titulo.innerHTML = "Novo título!";
titulo.style.color = "blue";
const paragrafos = document.querySelectorAll("p");
paragrafos.forEach(p => p.style.fontWeight = "bold");
```



Callbacks

Funções que recebem outras funções como argumento para execução futura.

```
// função que recebe callback
function processar(valor, callback){
  console.log("Processando", valor);
  callback(valor); // chama o callback com valor
}
processar(5, function(resultado){
  console.log("Resultado:", resultado); // imprime 5
});
```



Callback Hell

Quando usamos callbacks aninhados, fica difícil ler e manter.

```
login(usuario, senha, function(err, user){
  if(err) return console.error(err);
  buscarDados(user, function(err, dados){
    if(err) return console.error(err);
    atualizarUI(dados, function(err){
      if(err) return console.error(err);
      console.log("Tudo pronto!");
    });
  });
});
```

Solução: Promises ou Async/Await



Promises

```
function buscarUsuario(id){
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if(id > 0){
        resolve({id: id, nome: "Usuário " + id}); // sucesso
      } else {
        reject("ID inválido"); // erro
      }
    }, 1000); // simula 1s de delay
  });
}

// usando a promise
buscarUsuario(1)
  .then(usuario => console.log("Usuário encontrado:", usuario))
  .catch(err => console.error("Erro:", err));
```



Async / Await

```
async function obterUsuario(){
  try {
    const usuario = await buscarUsuario(2); // espera a Promise resolver
    console.log("Usuário via async/await:", usuario);
  } catch(err){
    console.error("Erro:", err);
  }
}

obterUsuario();
```



Assíncrono e chamadas ao backend

Chamadas assíncronas permitem que o código continue rodando enquanto espera a resposta de um servidor.

```
fetch("https://jsonplaceholder.typicode.com/posts")
  .then(res => res.json())
  .then(dados => console.log(dados)) // dados recebidos do backend
  .catch(err => console.error(err)); // trata erros
```

Tratamento de exceção

```
try {
  let resultado = 10 / 0; // só exemplo
  console.log(resultado);
} catch(err){
  console.error("Ocorreu um erro:", err);
} finally {
  console.log("Sempre executa");
}
```

Exercícios

Para testar, use o console do navegador ou arquivos HTML + JS.

Básico

1. Declarar variáveis com `let` e `const`.
2. Declarar um array de números e imprimir no console.
3. Criar função que soma dois números.
4. Alterar texto de um `<h1>`.
5. Adicionar evento de click a um botão.
6. Criar função que recebe um callback e chama-o.

Intermediário

7. Criar uma Promise que resolve com "Olá Mundo".
8. Criar Promise que rejeita se número for negativo.
9. Usar `async/await` para chamar a Promise acima.
10. Fazer `fetch` de API e imprimir resultado.
11. Tratar exceção usando `try/catch`.

● Avançado

12. Criar função assíncrona que chama duas Promises sequenciais e soma resultados.
13. Criar array de Promises e usar `Promise.all`.
14. Simular chamada ao backend que falha e tratar com `catch`.
15. Exemplo de callback hell e refatoração usando Promises.
16. Criar função que lê array e retorna Promise para cada elemento.
17. Exercício EventHub: criar função assíncrona que busca evento do "backend" e atualiza contagem de inscritos usando `await`.

🏆 Desafio — Busca de CEPs

Crie uma aplicação que:

- Possua um input para o usuário digitar um CEP.
- Possua um botão "Buscar" que faz a requisição à API ViaCEP (<https://viacep.com.br/ws/CEP/json/>).
- Mostre os resultados em cards acumulativos na tela, cada CEP buscado adiciona um card.
- 💡 Desafio Master: Adicione validação do CEP (somente números, tamanho correto), máscara visual e spinner enquanto a API responde.

📌 Gabarito dos Exercícios

● Básico

```
1. let x=10; const y=20;
2. let arr=[1,2,3]; console.log(arr);
3. function soma(a,b){return a+b;}
4. document.querySelector("h1").innerHTML="Novo título";
5. document.querySelector("button").addEventListener("click",()=>alert("Clique!"))
6. function testar(cb){cb("ok");} testar(console.log);
```



● Intermediário

```
7. new Promise(resolve=>resolve("Olá Mundo")).then(console.log);
8. new Promise((resolve,reject)=>{let n=5; n>=0?resolve("ok"):reject("negativo");
```

```
9. async function teste(){ try{ const res=await new Promise((res,rej)=>res("ok"))
10. fetch("https://jsonplaceholder.typicode.com/todos/1").then(res=>res.json()).t
11. try{ JSON.parse("erro"); } catch(e){ console.error(e); }
```

● Avançado

```
12. function p1(){ return Promise.resolve(2); }
    function p2(){ return Promise.resolve(3); }
    async function somaAsync(){ const a=await p1(); const b=await p2(); console.l
13. Promise.all([Promise.resolve(1),Promise.resolve(2)]).then(res => console.log(
14. function falha(){ return new Promise((res,rej)=>rej("erro")); } falha().catch
15. Usar Promises encadeadas para substituir callbacks aninhados.
16. const arr=[1,2,3]; arr.forEach(n => Promise.resolve(n).then(console.log));
17. async function inscrever(id){ const evento = await fetch(`https://api.exemplo
```

-
- ✓ Hoje aprendemos Callbacks, Promises, Async/Await, programação assíncrona, tratamento de exceções e exercícios do básico ao avançado com gabarito.